

Circles — System Design & Architecture Documentation

Assignment 3

17 April 2026

SUBMITTED TO

Software Development Case Study · CSE-2507M

SUBMITTED BY

Alexandr Tyulkov

Contents

1	Project Scope	1
2	Task 1 — Non-Functional Requirements & Trade-offs	1
2.1	1.1 NFR targets	1
2.2	1.2 Primary trade-off	2
2.3	1.3 Architectural consequence	2
3	Task 2 — High-Level Architecture Diagram (C4)	3
3.1	Method	3
3.2	2.1 Level 1 — System Context	3
3.3	2.2 Level 2 — Container Diagram	4
4	Task 3 — Architecture Decision Record	4
4.1	ADR-001 — Use a modular monolith with asynchronous background workflows	4
5	Task 4 — Detailed Component Design	6
5.1	Component interaction view	6
5.2	4.1 Web Frontend	6
5.3	4.2 API Server	7
5.4	4.3 Workflow Worker	7
5.5	4.4 PostgreSQL	8
5.6	4.5 Object Storage	8
6	Task 5 — UML Diagrams	9
6.1	5.1 Class Diagram	9
6.2	5.2 Sequence Diagram	10
6.3	5.3 Entity-Relationship Diagram	11
7	Architecture Assessment	11
7.1	Strengths	11
7.2	Known limitations	11
7.3	Recommended next architectural step	12
8	References to Methods	12

1 Project Scope

Circles is a web application for Spotify users who want a permanent record of listening history, social comparison, playlist tooling, and AI-generated music identity summaries. The implemented architecture is a modular monolith: a React frontend calls a Hono API, the API persists canonical state in PostgreSQL, and long-running jobs execute in a separate Hatcher worker.

This document describes the architecture that exists in the repository today and the rationale behind its main structural decisions. The system description is grounded in the current codebase: `apps/frontend`, `apps/backend`, the PostgreSQL schema in `apps/backend/src/db/postgres/schema.ts`, and the Hatcher workflows in `apps/backend/src/worker/workflows`.

2 Task 1 — Non-Functional Requirements & Trade-offs

2.1 1.1 NFR targets

Quality attribute	Target	Measurement method	Justification
Latency	P95 $\leq$ 200 ms for read APIs; P99 $\leq$ 500 ms	k6 load test on <code>/me/stats</code> , <code>/library/*</code> , <code>/leaderboard</code>	The dashboard and library views are exploratory. Responses that exceed 200 ms start to feel sluggish when users change ranges or paginate.
Availability	99.5% monthly uptime	Uptime monitoring against <code>/health</code>	This is sufficient for a course-work system with one production region and a small user base, while still requiring health checks and disciplined recovery.
Throughput	100 RPS sustained, 300 RPS burst on read traffic	Synthetic benchmark with mixed authenticated GET requests	Most traffic is read-heavy dashboard usage. This target supports class-demo scale without forcing premature microservices.
Consistency	Strong consistency for auth, follows, playlists, imports; eventual consistency for synced Spotify data, AI cache, leaderboard cache	Per feature and write path	User-owned data must reflect writes immediately, while derived analytics and cached views can tolerate bounded staleness.

Quality attribute	Target	Measurement method	Justification
Data durability	RPO < 24 h, RTO < 2 h	Backup re-store drill and import-job replay test	Listening history and social data are important, but the architecture does not justify multi-region replication for this scope. Daily recoverability is the practical target.

2.2 1.2 Primary trade-off

The dominant architectural choice is **operational simplicity and consistency over independent component scalability**. Circles keeps one canonical relational store and one API deployment unit, then offloads only slow or bursty work to asynchronous Hatchet workflows.

Trade-off axis	What the architecture optimizes for	What it gives up
Latency vs Throughput	Low latency for user-facing reads by keeping synchronous flows short and moving sync/import work to the worker	Peak write throughput is lower than a partitioned event-driven design because PostgreSQL remains the system of record.
Consistency vs Availability	Strong consistency for user actions such as follow/unfollow, playlist edits, and profile privacy	The system accepts reduced availability during a database outage because no eventually-consistent fallback store exists for critical writes.
Cost vs Performance	Lower operational cost and simpler maintenance using a modular monolith, one relational database, and limited caching	Less horizontal isolation than a microservice estate with dedicated stores and independent autoscaling.

2.3 1.3 Architectural consequence

These NFRs directly explain three implementation choices in the repository:

- PostgreSQL is the authoritative store for users, sessions, follows, playlists, history, AI cache metadata, and import jobs.
- Background workflows handle token refresh, Spotify sync, ZIP import, artist hydration, and weekly playlist generation so the API stays responsive.
- Only non-critical read paths use relaxed consistency, such as the 5-minute in-memory leaderboard cache and weekly AI result caching.

3 Task 2 — High-Level Architecture Diagram (C4)

3.1 Method

The diagrams follow the **C4 model** by Simon Brown. Level 1 shows the system in its environment; Level 2 shows the deployable containers and their protocols.

3.2 2.1 Level 1 — System Context

C4 Level 1 — System Context

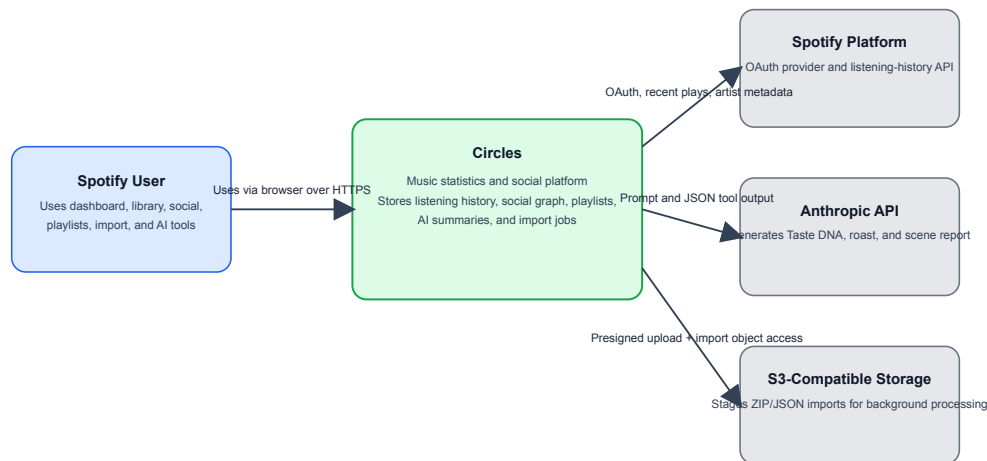


Figure 1: C4 Level 1 system context for Circles

The main actor is the Spotify user accessing Circles from a browser. The system depends on Spotify for OAuth and listening-history data, Anthropic for AI-generated reports, and S3-compatible object storage for ZIP import staging.

3.3 2.2 Level 2 — Container Diagram

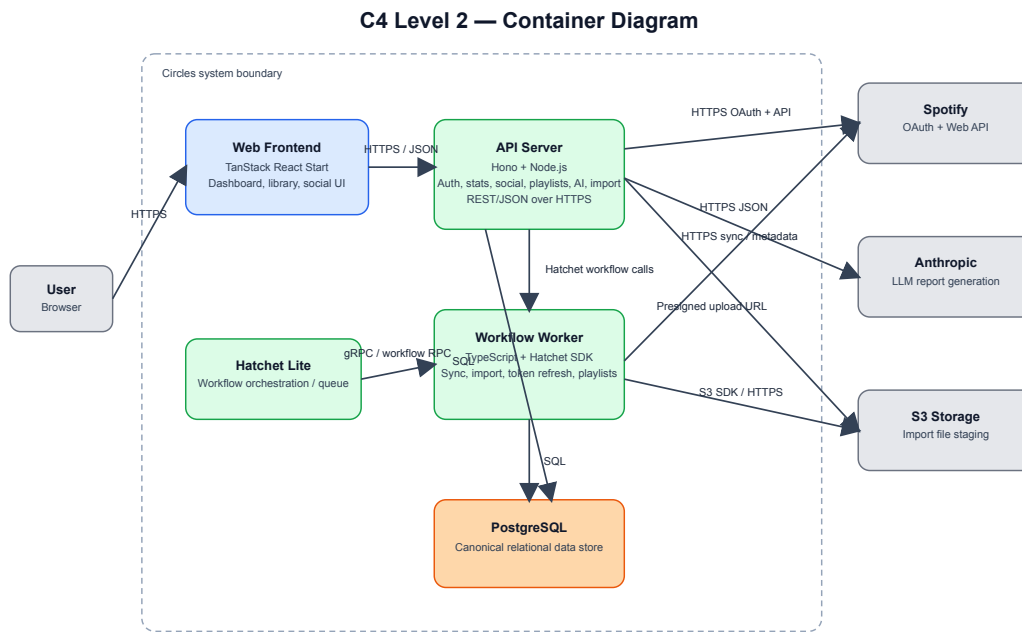


Figure 2: C4 Level 2 container diagram for Circles

The major containers are:

- **Web Frontend** — TanStack React Start application running in the browser and deployed via Cloudflare Workers.
- **API Server** — Hono-based Node.js service exposing auth, stats, library, social, playlist, AI, and import endpoints.
- **Workflow Worker** — Hatchet worker that executes slow or scheduled jobs.
- **PostgreSQL** — system of record.
- **Hatchet Lite** — workflow orchestrator / queue runtime used by the worker.

External services remain outside the system boundary and are accessed over HTTPS.

4 Task 3 — Architecture Decision Record

4.1 ADR-001 — Use a modular monolith with asynchronous background workflows

Field	Content
Status	Accepted
Date	17 April 2026
Context	Circles must support OAuth login, history sync from Spotify, playlist generation, social features, AI summaries, and ZIP imports. Some of these operations are user-facing and latency-sensitive; others are slow, bursty, or scheduled. The team size is one developer, the delivery window is a semester, and the NFRs favor strong consistency, acceptable read latency, and low operational overhead rather than massive scale.

Field	Content
Decision	Implement Circles as a modular monolith: one frontend application, one Hono API service, one PostgreSQL database, and one separate Hatchet worker process for asynchronous and scheduled workloads. Keep PostgreSQL as the single source of truth for operational data.
Rationale	This choice best satisfies the NFR profile. Strongly consistent playlist edits, follows, imports, and profile changes are simplest with one canonical relational store. Read latency stays low because the API handles only short synchronous work, while Hatchet absorbs token refresh, Spotify synchronization, ZIP processing, and playlist generation. Operationally this is much cheaper and more teachable than a multi-service architecture.
Alternatives rejected	re- 1. Microservices with separate stores — rejected because the project scale does not justify service discovery, distributed tracing, schema duplication, and failure coordination. 2. Single-process monolith with no background worker — rejected because ZIP imports, Spotify sync, and AI generation would either block request threads or require fragile ad hoc cron logic. 3. Event-sourced / queue-first analytics pipeline — rejected because it improves replayability and write throughput, but adds major complexity before the product has evidence that PostgreSQL cannot satisfy the workload.
Consequences	Easier: development speed, reasoning about transactions, schema evolution, and local testing. Harder: the database remains a central dependency, and API plus worker releases still need coordination around shared schema changes. New risks: backlog growth in Hatchet during external API outages and limited horizontal scaling for write-heavy futures.

5 Task 4 — Detailed Component Design

5.1 Component interaction view

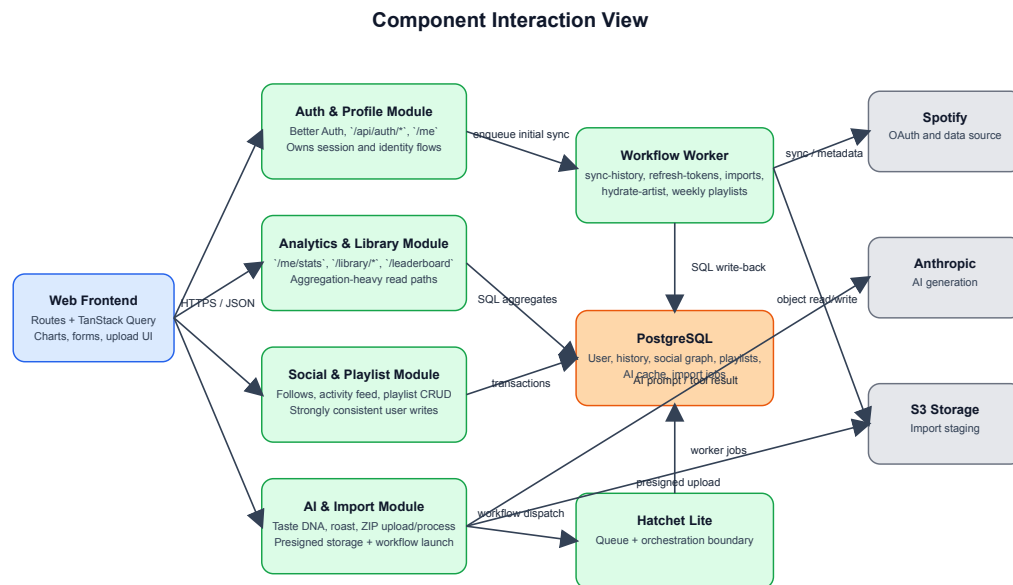


Figure 3: Component interaction view derived from the Level 2 containers

5.2 4.1 Web Frontend

Field	Description
Component name	Web Frontend
Purpose	Render the user interface and orchestrate user-driven requests to the API.
Responsibilities	• Handle navigation across dashboard, library, social, playlist, import, and AI routes. • Trigger authenticated API calls through the Hono client. • Cache query results in the browser via TanStack Query. • Render charts, tables, and upload workflows.
Inputs	HTTPS JSON responses from the API, OAuth callback redirects, and user interactions such as range selection or ZIP upload. Example input: { topArtists: [...], topTracks: [...] } for /me/stats?range=30d.
Outputs	HTTPS requests to the API. Example output: POST /me/playlists with { "name": "Road Trip", "description": "April set" }. Error case: 401 response causes redirect to login; 4xx/5xx responses produce user-visible failures.
Dependencies	Calls the API Server over HTTPS; depends indirectly on Spotify OAuth through redirect flows managed by the API.

Field	Description
NFR sensitivity	Latency. The frontend is constrained most by the requirement that dashboard and library interactions feel fast.

5.3 4.2 API Server

Field	Description
Component name	API Server
Purpose	Own synchronous business logic, validate requests, enforce auth, and expose the system contract.
Responsibilities	• Serve auth, profile, stats, social, playlist, AI, import, and leaderboard endpoints. • Enforce authentication and visibility rules. • Translate HTTP requests into model operations and workflow submissions. • Generate presigned S3 upload URLs and short-lived derived responses.
Inputs	JSON requests and cookies from the frontend. Example: { "s3Key": "imports/u123/1713340000.bin" } for POST /me/import/process.
Outputs	JSON documents and HTTP status codes. Example success: 201 { "jobId": "42" }. Error case: 404 { "message": "Playlist not found" } or 403 { "message": "Profile is private" }.
Dependencies	Reads and writes PostgreSQL; enqueues work in Hatchet Lite; calls Spotify, Anthropic, and S3 when synchronous behavior requires it.
NFR sensitivity	Latency and consistency. The API must keep request paths short while preserving correct user-visible writes.

5.4 4.3 Workflow Worker

Field	Description
Component name	Workflow Worker
Purpose	Execute slow, scheduled, and retryable background jobs outside the request path.
Responsibilities	• Sync recent plays from Spotify after login and on schedule. • Refresh expiring Spotify tokens. • Process ZIP imports in batches. • Hydrate artist metadata and generate weekly playlists.
Inputs	Workflow payloads from Hatchet Lite. Example: { "userId": "u123", "jobId": "42", "s3Key": "imports/u123/1713340000.bin" }.

Field	Description
Outputs	Database mutations, S3 reads/writes, and workflow status logs. Error case: mark import job as failed with errorMessage.
Dependencies	Consumes jobs from Hatchet Lite; reads and writes PostgreSQL; calls Spotify APIs and S3 object storage.
NFR sensitivity	Throughput. The worker must absorb bursty sync and import workloads without degrading the API.

5.5 4.4 PostgreSQL

Field	Description
Component name	PostgreSQL
Purpose	Persist all canonical operational data and support relational queries for analytics and social features.
Responsibilities	• Store auth/session data, users, follows, playlists, history, AI cache metadata, and import jobs. • Enforce primary keys, foreign keys, uniqueness, and indexes. • Support aggregate read queries for dashboards and library views. • Provide durable state shared by API and worker processes.
Inputs	SQL reads/writes from the API and worker. Example insert into history: (user_id, track_id, played_at).
Outputs	Relational rows and aggregate query results. Error case: unique-key conflict on duplicate history entry is safely ignored through onConflictDoNothing.
Dependencies	Called by API Server and Workflow Worker. No downstream dependencies inside the system.
NFR sensitivity	Consistency and durability. PostgreSQL is the main source of truth, so its design is constrained most strongly by correctness and recoverability.

5.6 4.5 Object Storage

Field	Description
Component name	Object Storage
Purpose	Temporarily stage uploaded Spotify export files and normalized import payloads.
Responsibilities	• Accept browser uploads through presigned URLs. • Store original import binaries and derived normalized JSON.

Field	Description
	• Provide worker-readable objects for batch import processing.
Inputs	Binary ZIP or JSON uploads from the browser; normalized JSON from the worker. Example: ZIP containing Streaming_History_Audio_2024*.json.
Outputs	Object bytes returned to the worker; presigned upload destination for the frontend. Error case: expired signed URL or missing object returns 403/404 from S3-compatible storage.
Dependencies	Called by the frontend through signed URLs and by the worker/API through the S3 SDK.
NFR sensitivity	Throughput. Import spikes should be buffered in storage rather than forcing large request bodies through the API server.

6 Task 5 — UML Diagrams

6.1 5.1 Class Diagram

UML Class Diagram

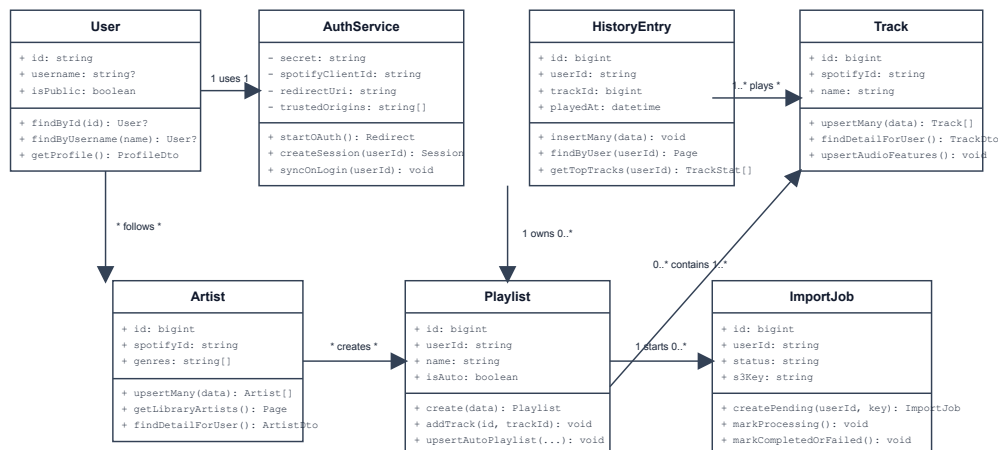


Figure 4: UML class diagram for the main domain and service classes

The class view combines persistent entities with one service object (**AuthService**) because login and session creation are central to the runtime behavior of the system.

6.2 5.2 Sequence Diagram

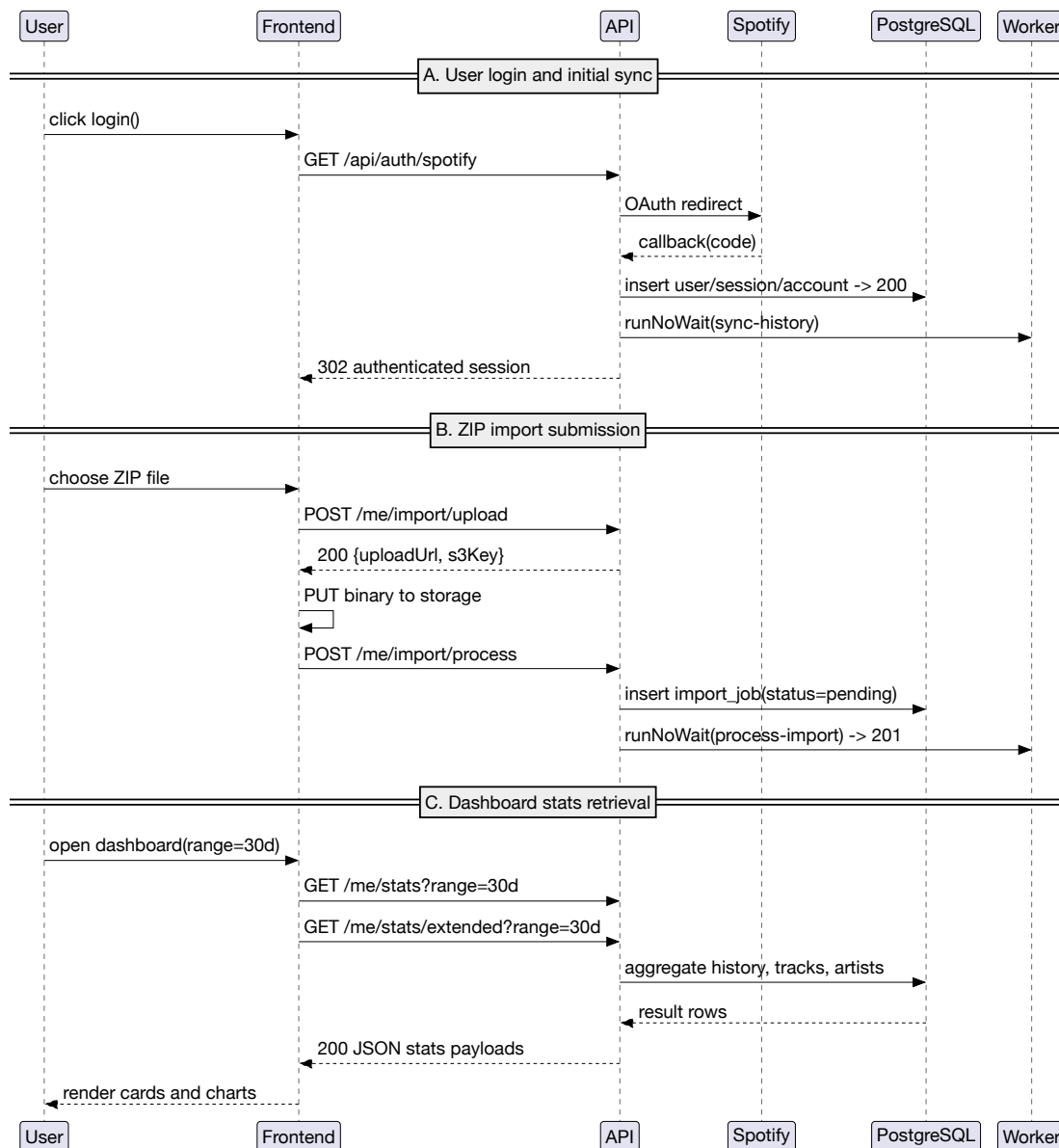


Figure 5: Three key sequence flows: login, ZIP import submission, and dashboard stats retrieval

The third scenario is dashboard statistics retrieval, which is specific to Circles and demonstrates how the system serves exploratory analytics without routing the user through background infrastructure.

6.3 5.3 Entity-Relationship Diagram

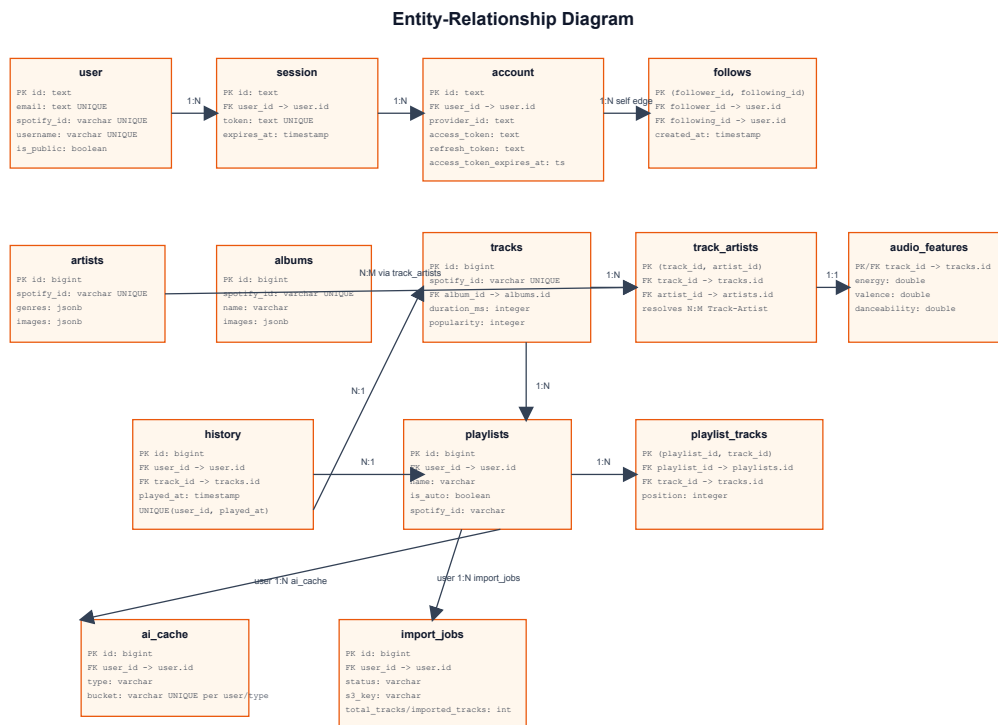


Figure 6: Entity-relationship diagram for the PostgreSQL schema

The ER model resolves every many-to-many relationship explicitly:

- `track_artists` resolves `Track` $\leftrightarrow$ `Artist`.
- `playlist_tracks` resolves `Playlist` $\leftrightarrow$ `Track`.
- `follows` resolves `User` $\leftrightarrow$ `User`.

This matches the implemented schema and preserves relational integrity for analytics, social features, and playlist operations.

7 Architecture Assessment

7.1 Strengths

- The architecture is coherent with the semester scope and one-developer constraint.
- PostgreSQL fits the data model well because Circles mixes transactional data, social graph edges, and aggregate analytics over shared entities.
- Hatchet cleanly separates slow work from the API path.
- The system remains understandable because the frontend, API, worker, and schema all live in one repository.

7.2 Known limitations

- PostgreSQL is a single critical dependency; there is no read replica or failover node in the current design.
- The leaderboard cache is in-process, so cache state is not shared across multiple API replicas.

- AI calls remain synchronous from the API and therefore can affect request latency if the provider is slow.
- Import processing depends on both S3-compatible storage and Hatchet availability.

7.3 Recommended next architectural step

If Circles grows beyond coursework scale, the highest-value improvement is not microservices. It is **infrastructure hardening around the existing shape**: shared cache for leaderboard and AI metadata, PostgreSQL backups with restore validation, stronger workflow observability, and possibly moving AI generation onto the worker if user demand makes those requests frequent.

8 References to Methods

- **C4 Model** — Simon Brown. Used for system context and container views.
- **Architecture Decision Record (ADR)** — Michael Nygard. Used to document the central structural decision.
- **Unified Modeling Language (UML)** — used for class and sequence diagrams.
- **Entity-Relationship Modeling** — used to document the relational schema and cardinalities.